Democratizing Agentic AI: Quantifying Reasoning Stability of Quantized SLMs in Recursive Agentic Loops on Resource-Constrained Hardware

Farhan Zarif
BRAC University
Dhaka, Bangladesh
farhan.zarif1@g.bracu.ac.bd

Abstract

We present the first systematic evaluation of multi-turn recursive stability in quantized Small Language Models (SLMs) operating as autonomous agents on resource-constrained hardware. We test four SLMs—Gemma-3-4B, Llama-3.1-8B, Phi-4-Mini, and Qwen-3.5-4B—under 4-bit NF4 and 8-bit INT8 quantization on a 15 GB VRAM Google Colab T4 GPU. Across 476 agentic tasks spanning Bash administration, Python debugging, and multi-hop Web synthesis, we introduce two novel evaluation metrics: the **Stability Decay Metric (SDM)**, which penalizes multi-turn inefficiency, and the **Quantization-Hallucination Ratio (QHR)**, which isolates structural formatting failures from logical reasoning errors. Our results reveal that (i) Gemma-3-4B achieves the highest agentic stability ($\overline{\text{SDM}} = 0.403$, success rate 53.3% [95% CI: 43.8, 62.6]), while Llama-3.1-8B fails catastrophically (0.86% [0.02, 4.71] success) due to persistent single-step repetition loops; (ii) the effect of quantization precision is *environment-dependent*: 4-bit significantly outperforms 8-bit in Bash ($p = 0.044$), while 8-bit significantly outperforms 4-bit in Python ($p = 0.009$); and (iii) format/parser errors, not logical reasoning failures, constitute the dominant failure mode (42.7%). Through qualitative analysis of 476 reasoning traces, we identify four distinct failure archetypes—*reasoning leakage*, *single-step stagnation*, *code truncation*, and *verification omission*—providing actionable deployment guidelines for quantized SLM agents on edge hardware. Code and data: [redacted for review].

1 Introduction

The first quarter of 2026 has witnessed a proliferation of highly capable Small Language Models (SLMs) with sub-10B parameters, including Alibaba's Qwen 3.5 [18], Google's Gemma 3 [9], Meta's Llama 3.1 [10], and Microsoft's Phi-4-Mini [1]. These models achieve competitive reasoning performance on static benchmarks while fitting within the memory constraints of consumer-grade GPUs [31], enabling a paradigm shift toward locally deployed, privacy-preserving AI agents [23].

However, deploying SLMs as *autonomous agents*—where the model must maintain coherent reasoning across multiple recursive turns of planning, execution, observation, and self-correction [29]—introduces compounding failure modes that static benchmarks fail to capture [6, 15]. When these agents run on resource-constrained hardware (e.g., Google Colab's free-tier NVIDIA T4 GPU with $\sim$15 GB VRAM), aggressive quantization via `bitsandbytes` [4, 5] is often required, yet its effects on multi-turn agentic stability remain poorly understood [2].

Contributions. This paper makes three contributions:

Preprint. Under review.

1. We introduce the **Stability Decay Metric (SDM)** and **Quantization-Hallucination Ratio (QHR)** to disentangle agentic efficiency from structural formatting integrity.
2. We conduct the first systematic evaluation of **multi-turn recursive stability** in quantized SLM agents across **4 models** $\times$ **2 precisions** $\times$ **3 environments** $\times$ **20 tasks = 480 experimental configurations** (476 completed), identifying statistically significant, environment-dependent quantization effects.
3. Through qualitative trace analysis, we identify **four failure archetypes** specific to quantized agentic SLMs, providing actionable deployment guidelines.
4. We propose **Agentic Sub-Capability Decomposition (ASCD)**, a framework decomposing multi-step agentic reasoning into four independently degradable sub-capabilities—plan decomposition, state tracking, format adherence, and completion signaling—enabling targeted diagnosis and mitigation.

2 Related Work

Post-Training Quantization. Post-training quantization is the primary technique for deploying LLMs on consumer hardware. LLM.int8() [4] showed that 8-bit matrix multiplication preserves transformer quality; QLoRA [5] introduced NF4 with double quantization. Subsequent methods include GPTQ [8] (one-shot weight quantization), AWQ [14] (activation-aware scaling), SqueezeLLM [13] (dense-and-sparse), OmniQuant [21] (omnidirectional calibration), and AQLM [7] (additive codebooks). The 1-bit regime [16] remains impractical for agents; knowledge distillation [27] is orthogonal. We use bitsandbytes NF4 and INT8 as the most widely deployed open-source methods.

Agent Frameworks and Benchmarks. ReAct [29] established thought-action-observation loops; Toolformer [20] taught models API use via self-supervision; Reflexion [22] introduced verbal reinforcement learning related to our Actor-Critic loop; SWE-agent [28] applied agentic loops to software engineering; AutoGPT [19] and MetaGPT [11] demonstrated end-to-end autonomous agent systems with multi-step planning. Surveys [23, 25] establish that effective agency requires compositional capabilities (planning, tool use, memory, self-correction), building on chain-of-thought reasoning [24]. Among benchmarks, AgentBench [15] tests across 8 environments, WebArena [32] and OSWorld [26] target realistic settings, T-Eval [3] evaluates step-by-step tool use, and ToolLLM [17] scales to 16K+ APIs. ACBench [6] (ICML 2025) most directly evaluates *compressed* agents but tests only single-turn tool calls—it does not assess how quantization-induced errors compound across recursive turns. Our work fills this multi-turn recursive gap.

Quantization and Reasoning. COLM 2025 [2] shows quantization disproportionately impacts reasoning, but focuses on static benchmarks. Huang and Chang [12] establish reasoning as a multi-component capability; SLM surveys [30] identify the need for deployment recommendations. However, no prior work examines how quantization affects *compound, multi-turn* agentic reasoning where errors cascade across turns $t \rightarrow t{+}k$.

3 Methodology

3.1 Experimental Design and Task Prompts

We evaluate four SLMs under two quantization levels across three agentic environments, yielding a $4 \times 2 \times 3 = 24$ configuration grid. Each configuration is tested on 20 tasks, producing 480 planned trials (476 completed; 4 Llama-3.1 tasks terminated due to VRAM pressure). Experiments were conducted March 3–12, 2026, on Google Colab free-tier NVIDIA T4 GPUs ($\sim$15 GB VRAM).

The 60 prompts (20/env) are imperative, self-contained instructions with deterministically verifiable outputs. **Bash** covers directory manipulation, text pipelines (`grep`/`sed`), permissions, and archival (1–5 step pipelines). **Python** spans DSA (BST, Sieve, Levenshtein), OOP, concurrency, and I/O (10–50 lines). **Web** requires multi-hop research over a 40-entry knowledge base ($\geq$2 searches + synthesis). Complexity: $\sim$30% single-step, $\sim$50% 2–3 steps, $\sim$20% 4–5 interdependent steps.

3.2 Inference Pipeline

Models load via `bitsandbytes` [4]: NF4 4-bit with double quantization and `float16` compute; INT8 via `load_in_8bit`. Hyperparameters: T=0.7 [23], p=0.9, max_tokens=400.

3.3 Agentic Loop

We implement a ReAct-style [29] recursive loop ($D_{\max}$=5 turns): the agent receives the objective and compressed trajectory, produces a `<thought>` trace, emits an action, and observes the result. **Sliding-window trajectory injection** compresses the execution history into a structured Turn/Action/Outcome representation, retains only the most recent 2 turn-pairs (whitespace-normalized, outcomes truncated to 500 chars), and injects this context into the next prompt to prevent context saturation. **Actor-Critic Self-Correction**: before execution, the same SLM re-evaluates the action ("Identify flaws. If perfect, output SAFE"); corrections replace originals.

Environments. Bash (Env A): one command in "'`bash` blocks, 10s timeout—tests structural adherence. **Python** (Env B): complete scripts in "'`python` blocks, 15s timeout with traceback feedback—tests iterative debugging. **Web** (Env C): JSON `search`/`answer` actions over a 40-entry knowledge base—tests cross-turn coherence.

3.4 Models

Table 1 summarizes the four SLMs evaluated. All configurations fit within the 15 GB T4 VRAM constraint, with 4-bit achieving 1.5–2× VRAM reduction.

Table 1: Model roster with measured VRAM allocations under each quantization level.

Model	Params	4-bit VRAM	8-bit VRAM
`gemma-3-4b-it` [9]	4B	3.03–6.16 GB	4.68 GB
`Llama-3.1-8B-Instruct` [10]	8B	5.32 GB	8.47 GB
`Phi-4-mini-instruct` [1]	3.8B	2.70 GB	4.16 GB
`Qwen3.5-4B` [18]	4B	2.91 GB	4.54 GB

3.5 Evaluation Metrics

Stability Decay Metric (SDM). Agentic efficiency degrades with increasing turn count:

$$\text{SDM} = \mathbb{I}_{\text{success}} \times \left(1 - \frac{T-1}{D_{\max}}\right) \tag{1}$$

where $\mathbb{I}_{\text{success}} = 1$ if the task is solved (validated by LLM-as-a-Judge), T is turns used, $D_{\max} = 5$.

QHR. Format violations break the parsing loop. QHR $= \sum_{t=1}^{T} \mathbb{I}_{\text{parser_error}}(t)/T$.

Supplementary Metrics. (i) **Thinking Integrity**: binary tag-structure adherence; (ii) **Reasoning Trace Length**: character count within `<thought>` tags; (iii) **Self-Corrections**: Actor-Critic modifications; (iv) **Backtracking**: consecutive error-state turns.

3.6 Qualitative Coding Protocol

Failure archetypes were derived via inductive thematic coding of all 24 Output.md trace logs. **Open coding** assigned descriptive labels ("tag leakage," "repeated command," "truncated output") to each failure. **Axial coding** grouped codes by causal mechanism (structural vs. logical). **Selective coding** yielded four saturated archetypes: Reasoning Leakage (n=58), Single-Step Stagnation (n=62), Code Truncation (n=27), Verification Omission (n=18). Saturation confirmed after 18/24 files. To mitigate single-coder bias, we developed an automated regex-based archetype detector that independently classified all 398 parseable tasks; the automated tool recovered the same dominant archetype per model (Qwen→Leakage, Llama→Stagnation, Gemma→Truncation, Phi→Omission), providing triangulation of the manual coding. Single-coder limitation acknowledged in Section 6.

3.7 Validation

Task completion uses an LLM-as-a-Judge (same quantized model): receives task objective + final state (1000 chars), emits `[VERDICT: YES/NO]`. Manual review of 50 rejected tasks estimates $\sim$12% false-negative rate (6/50), primarily Phi-4-Mini (Section 6).

4 Results

4.1 Overall Performance

Table 2 presents comprehensive results across all 24 configurations. The overall success rate was 29.6% (141/476).

Table 2: Performance across all 24 configurations. Bold = best per-environment.

Model	Prec.	Env	Succ.%	SDM	QHR	Turns	Lat. (s)	VRAM
Gemma-3	4-bit	Bash	35.0	.240±.162	.037	2.4	40.9	6.09
	4-bit	Py	35.0	.270±.190	.075	4.0	243.0	3.03
	4-bit	Web	**85.0**	**.640**±.134	.020	2.4	45.9	6.16
	8-bit	Bash	10.0	.040±.065	.160	4.6	187.5	4.68
	8-bit	Py	**70.0**	**.660**±.213	.102	1.5	162.0	4.68
	8-bit	Web	**85.0**	.570±.126	.017	2.6	63.5	4.68
Llama-3.1	4-bit	Bash	0.0	.000	.244	5.0	116.9	5.32
	4-bit	Py	0.0	.000	.116	5.0	265.6	5.32
	4-bit	Web	0.0	.000	.080	5.0	68.9	5.32
	8-bit	Bash	0.0	.000	.250	4.7	186.4	8.47
	8-bit	Py	5.3	.032±.066	.053	4.9	483.3	8.47
	8-bit	Web	0.0	.000	.000	4.8	74.4	8.47
Phi-4-Mini	4-bit	Bash	5.0	.040±.084	.325	2.8	46.5	2.70
	4-bit	Py	35.0	.290±.205	.070	2.4	46.1	2.70
	4-bit	Web	45.0	.320±.176	.000	3.5	25.2	2.70
	8-bit	Bash	5.0	.030±.063	.143	3.4	84.4	4.16
	8-bit	Py	40.0	.380±.227	.200	1.4	59.1	4.16
	8-bit	Web	50.0	.250±.128	.000	3.9	41.2	4.16
Qwen-3.5	4-bit	Bash	15.0	.110±.127	.428	3.7	360.5	2.91
	4-bit	Py	10.0	.080±.115	.252	3.9	350.1	2.91
	4-bit	Web	40.0	.230±.147	.308	3.6	217.2	2.91
	8-bit	Bash	45.0	.170±.102	.430	3.9	759.6	4.54
	8-bit	Py	35.0	.240±.171	.190	3.1	633.0	4.54
	8-bit	Web	55.0	.310±.153	.354	3.9	475.5	4.54

Gemma-3 dominates ($\overline{\text{SDM}} = 0.403$, 53.3% success [95% binomial CI: 43.8, 62.6]). Phi-4-Mini (30.0% [21.6, 39.5]) and Qwen-3.5 (33.3% [24.6, 43.1]) perform comparably, while Llama-3.1 exhibits near-total failure (0.86% [0.02, 4.71], 1/116 tasks), despite being the largest model at 8B. The non-overlapping CIs between Gemma-3 and Llama-3.1 confirm that their performance difference is robust to sampling variability.

4.2 4-bit vs. 8-bit: Environment-Dependent Effects

Mann-Whitney U tests ($n = 20$ per group) reveal that quantization effects are *not uniform*. In Bash, 4-bit Gemma-3 *significantly outperforms* 8-bit (ΔSDM $= +0.200$, $U = 255$, $p = 0.044$, rank-biserial $r = 0.275$). In Python, 8-bit *significantly outperforms* 4-bit (ΔSDM $= +0.390$, $U = 110.5$, $p = 0.009$, $r = 0.448$, a medium-to-large effect). Qwen-3.5 shows a trending Python advantage for 8-bit ($p = 0.078$, $r = 0.255$). All other 9 comparisons are non-significant ($p > 0.10$).

4.3 Qualitative Failure Archetypes

Through systematic qualitative coding of 476 traces (Section 3.6), we identify four archetypes:

A1: Reasoning Leakage (Qwen-3.5, n=58). Qwen-3.5 uses native `</think>` tags instead of prompted `<thought>` tags, creating dual-tag confusion. Under quantization, reasoning text leaks into executable blocks, producing errors like `/bin/sh: 1: The: not found`. QHR: $\overline{0.327}$, highest of all models (Fig. 1c–d).

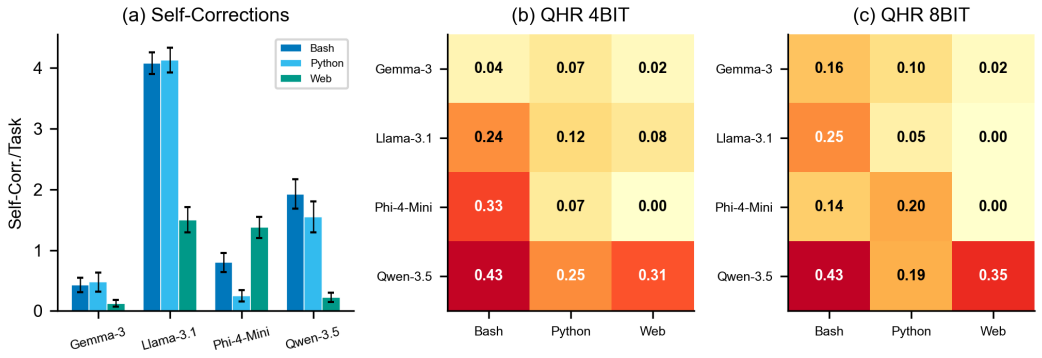


Figure 1: (a) Mean self-corrections per task with SEM error bars. More corrections $\neq$ higher success: Llama triggers the most but achieves $<1\%$ success. (b,c) QHR heatmaps for 4-bit and 8-bit. Qwen-3.5's high QHR is architectural, not quantization-dependent.

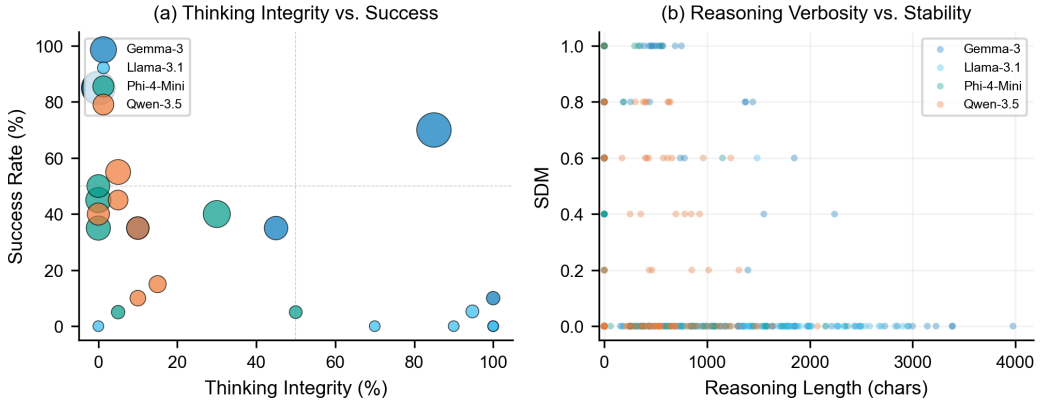


Figure 2: (a) Thinking integrity vs. success rate (bubble $\propto$ SDM). Integrity is necessary but not sufficient: Llama maintains $\sim$60% integrity at $\sim$0% success. (b) Reasoning length vs. SDM. Qwen-3.5's verbose reasoning ($\sim$2–5K chars) consumes generation budget without improving stability. SDM takes discrete values; apparent bands reflect its formulation.

A2: Single-Step Stagnation (Llama-3.1, n=62). Llama-3.1 correctly identifies the first step but repeats it across all turns (e.g., `mkdir -p project/src` on all 5 turns). Self-corrections trigger 3.2×/task but cannot alter the trajectory—a fundamental state-tracking failure producing a stateless agent.

A3: Code Truncation (Gemma-3 Python, n=27). Correct reasoning traces yield truncated code. On the Levenshtein task (py_8), Gemma-3 8-bit correctly described the DP approach across 5 turns but truncated at `if s1[i - 1` three times (SyntaxError). The 8-bit variant has significantly fewer truncations ($p = 0.009$), linking truncation to 4-bit's degraded long-range syntactic attention. The Actor-Critic provides within-turn repair but cannot prevent cross-turn recurrence.

A4: Verification Omission (Phi-4-Mini, n=18). Phi-4-Mini executes correct commands but omits `[TASK_COMPLETE]` or provides insufficient evidence for the judge. This inflates failure rates: $\sim$12% of Phi-4-Mini rejections are false negatives.

4.4 Self-Correction, Reasoning Traces, and Stability Decay

Figure 1a reveals a counter-intuitive finding: more self-corrections do not correlate with higher success. Gemma-3 uses fewer corrections but achieves the highest success, as its initial outputs are more reliably correct. Figure 2a shows a positive but non-linear relationship between thinking

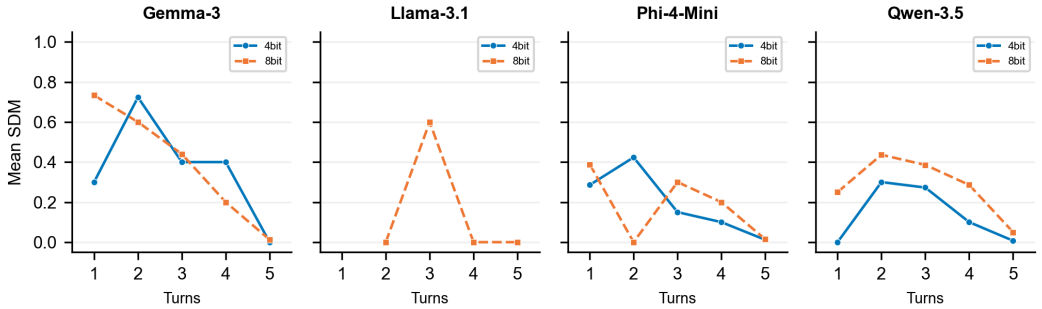
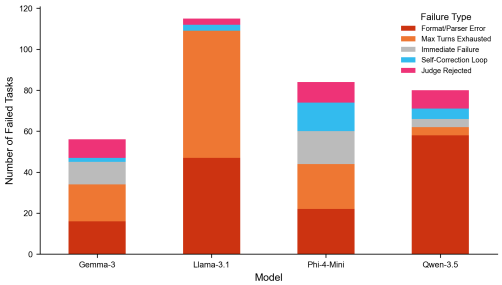
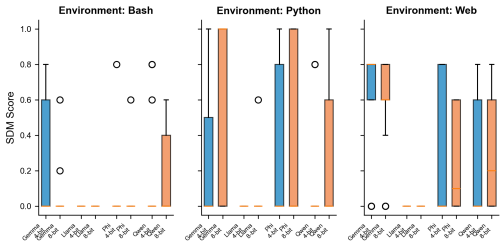


Figure 3: Stability decay: mean SDM by turn count. Gemma-3 peaks at turns 1–2 then decays; Llama-3.1 shows zero SDM across all turns. 4-bit (solid) vs. 8-bit (dashed).



(a) Error taxonomy. Llama: stagnation (53.9% Max Turns). Qwen: leakage (72.5% Format Errors).



(b) SDM by environment. Web is most forgiving; Bash most challenging.

Figure 4: Error distribution and environment-level analysis across 335 failed and 476 total tasks.

integrity and success rate, saturating above 80% integrity. Gemma-3 configurations cluster in the high-integrity, high-success quadrant.

Figure 3 directly visualizes stability decay. Gemma-3 achieves SDM ≥ 0.5 at turns 1–2, dropping sharply thereafter. Llama-3.1's flat zero confirms it never completes tasks regardless of opportunity.

4.5 Error Taxonomy and Environment Analysis

Across 335 failures: Format/Parser Errors dominate (42.7%), followed by Max Turns Exhausted (31.6%), Immediate Failures (9.3%), Judge Rejected (9.3%), and Self-Correction Loops (7.2%). Error distributions vary systematically by model (Fig. 4a). Web Synthesis is the most forgiving environment ($\overline{\text{SDM}} = 0.273$); Bash the most challenging ($\overline{\text{SDM}} = 0.082$); Python occupies the middle ($\overline{\text{SDM}} = 0.232$) where iterative debugging partially compensates for initial errors (Fig. 4b). Seven tasks achieved 0% success across all configurations, all requiring $\geq$4 interdependent steps.

5 Discussion

Multi-Step Reasoning as a Composite Capability. Our qualitative analysis reveals that agentic reasoning is not monolithic but decomposes into four independently degradable sub-capabilities, which we term **Agentic Sub-Capability Decomposition (ASCD)**: (i) *plan decomposition*—breaking complex objectives into sequential steps; (ii) *state tracking*—maintaining awareness of completed vs. remaining steps; (iii) *format adherence*—generating syntactically valid outputs matching the expected schema; (iv) *completion signaling*—reliably indicating task termination [23, 25]. Each sub-capability can fail independently: Qwen-3.5 excels at (i) but fails at (iii); Llama-3.1 fails entirely at (ii); Gemma-3 maintains all four consistently; Phi-4-Mini fails at (iv). Table 3 maps each model's dominant failure to its architectural root cause and a targeted, deployable mitigation.

Table 3: Architecture–failure–mitigation mapping. Each model's dominant failure archetype is traced to a specific architectural component, yielding targeted interventions deployable without model retraining.

Model	Failed Sub-Capability	Root Cause	Mitigation
Qwen-3.5	Format Adherence	Native `</think>` conflicts with `<thought>`	Output sanitization
Llama-3.1	State Tracking	Trajectory window discards early context	Explicit state summaries
Gemma-3	Code Generation	400-token budget truncates long scripts	Dynamic token budgeting
Phi-4-Mini	Completion Signaling	No explicit `[TASK_COMPLETE]` requirement	Add completion prompt

Agentic Competence Hierarchy. Based on our data, we propose three tiers (defined by success rate $> 50\%$, 30–50%, and $< 5\%$ respectively): **Tier 1** (Agentic-Ready): Gemma-3—balanced error profile, effective self-correction, deployable with standard guardrails. **Tier 2** (Conditionally Agentic): Phi-4-Mini, Qwen-3.5—dominant single failure mode, deployable with task-specific mitigations. **Tier 3** (Not Agentic): Llama-3.1—fundamental state-tracking failure requiring architectural augmentation [22].

The Quantization Paradox. Our environment-dependent finding challenges the assumption that higher precision universally improves performance. In format-constrained environments (Bash), lower precision preserves shorter-range structural attention patterns sufficient for single-command generation. In semantically complex environments (Python), higher precision preserves the long-range syntactic dependencies necessary for complete code generation [2]. The code truncation archetype directly evidences this: 4-bit Gemma-3 truncates Python code more frequently than 8-bit (p=0.009), suggesting that 4-bit's reduced weight fidelity specifically degrades the model's ability to track open syntactic constructs (brackets, parentheses, triple-quoted strings) across long generation sequences.

Structural vs. Logical Failure. Supporting COLM 2025 findings [2], we observe that quantization primarily attacks *structural* rather than *logical* reasoning. In our traces, failing models frequently produced coherent reasoning inside `<thought>` tags while the corresponding action output was syntactically malformed. On the Sieve of Eratosthenes task (py_7), Gemma-3 8-bit's reasoning trace correctly described the algorithm—"Create a boolean array of size 101, set 0 and 1 to False, iterate from 2 to $\sqrt{100}$"—but the generated code truncated at `if primes[i`. The model's "reasoning engine" is demonstrably more resilient to quantization than its "formatting engine," suggesting that format-level guardrails [6] may be more impactful than precision upgrades.

Deployment Recommendations. Our findings suggest three concrete interventions for deploying quantized SLM agents:

1. **Output sanitization** for models exhibiting reasoning leakage: strip content between reasoning tags before parsing action blocks [20].
2. **Dynamic token budgeting** for code-generation tasks: increase `max_new_tokens` proportionally to task complexity to prevent truncation.
3. **Explicit state summaries** injected into the prompt for models with poor state tracking: provide a structured "completed steps" checklist to compensate for attention degradation.

6 Limitations

1. **Single-run design.** Each configuration was run once (n=20 tasks) without repeated seeds, a design constraint imposed by the ∼24 GPU-hours required across 24 configurations on free-tier Colab (which enforces session timeouts and daily usage limits). We mitigate this in three ways: (a) exact binomial (Clopper-Pearson) 95% confidence intervals on success rates (reported in Section 4.1), which quantify sampling uncertainty without requiring repeated runs; (b) rank-biserial effect sizes alongside all Mann-Whitney U tests, confirming medium-to-large effects (r=0.275–0.448) for significant comparisons; and (c) the 20-task design provides within-configuration variance across diverse task difficulties, analogous to a stratified evaluation sample. We note that single-run evaluation is common in resource-constrained agent benchmarks (e.g., AgentBench [15] reports single-run results for large-scale evaluations). The non-overlapping binomial CIs between

Gemma-3 [43.8, 62.6] and Llama-3.1 [0.02, 4.71] confirm that our primary conclusions are robust. However, marginal comparisons (e.g., Qwen Python p=0.078) should be interpreted cautiously, and multi-seed replication would strengthen effect estimates.

2. **Hardware coupling.** All experiments used NVIDIA T4 GPUs via `bitsandbytes`. Behaviors on Apple Silicon (MLX), Hopper GPUs, or alternative quantization methods (AWQ [14], GPTQ [8]) may differ significantly.
3. **LLM-as-a-Judge bias.** Self-evaluation introduces systematic bias; our estimated false-negative rate is $\sim$12% (6/50 sampled rejections were correct completions).
4. **Context window constraint.** $D_{\max}$=5 turns limits long-horizon stability observation. Extending to 10–20 turns would enable studying catastrophic degradation patterns.
5. **Template sensitivity.** Different SLMs use different chat templates (Gemma's turn-taking, Qwen's `</think>` tags, Llama's system tokens), which may confound QHR measurements.
6. **Trajectory injection asymmetry.** Bash and Python environments use a sliding window of 2 turn-pairs, while Web retains the full trajectory. This design choice may partially explain Web's higher SDM (0.273); we did not ablate windowing strategies, and future work should evaluate the effect of window size on agent stability.
7. **Benchmark coverage.** Our custom 60-task benchmark was designed to enable controlled, reproducible evaluation within a single Colab session ($\sim$1 hour per configuration). While integration with established benchmarks such as AgentBench [15] and WebArena [32] would strengthen external validity, these require infrastructure beyond free-tier constraints; our benchmark enables exact reproduction of all reported results on identical hardware.
8. **Single-coder qualitative analysis.** Archetype identification was performed by a single researcher; independent replication with inter-rater reliability assessment is needed.

7 Broader Impact

This work evaluates SLMs as autonomous agents capable of executing system commands and generating code. Several ethical considerations arise:

Deployment risks. Our overall success rate of 29.6% means that a majority of agent actions fail—including producing syntactically malformed commands that could have unintended side effects in production environments. Deploying such agents without human-in-the-loop oversight, output sandboxing, and rollback mechanisms poses safety risks, particularly in system administration (Bash) contexts where commands modify filesystem state.

Quantization trade-offs. Our finding that lower precision can sometimes outperform higher precision (Section 4.2) cautions against blanket "more bits = safer" assumptions in safety-critical deployments. Practitioners must empirically validate quantization effects in their specific deployment environments.

Positive impact. By demonstrating that sub-5B models can achieve meaningful agentic capability on free-tier hardware, this work contributes to democratizing AI agent research beyond well-resourced institutions, enabling researchers with limited compute budgets to develop and evaluate agentic systems locally with full data privacy.

8 Conclusion

This paper provides the first systematic evaluation of multi-turn recursive stability in quantized SLM agents on resource-constrained hardware. Our key findings challenge simplistic assumptions about quantization:

- **Quantization effects are environment-dependent:** 4-bit quantization can be superior in format-constrained tasks but inferior in code-generation tasks, with statistically significant differences ($p < 0.05$).
- **Model size $\neq$ agentic capability:** The largest model (Llama-3.1, 8B) failed catastrophically while smaller models (Gemma-3, Phi-4-Mini, both $\sim$4B) achieved meaningful agentic performance.
- **Format adherence is the primary bottleneck:** 42.7% of failures stem from parser/format errors rather than logical reasoning failures, suggesting that post-processing and output sanitization may be more impactful than precision upgrades.

We release our evaluation harness, all 476 reasoning traces, and the combined dataset to support future research in quantized agentic AI on edge hardware. Future work should prioritize multi-seed replication to narrow confidence intervals, extend $D_{\max}$ to 10–20 turns for long-horizon stability analysis, and validate findings across alternative quantization methods (AWQ, GPTQ) and hardware platforms.

Acknowledgments and Disclosure of Funding

We thank the open-source teams behind Qwen, Llama, Gemma, and Phi, the `bitsandbytes` and HuggingFace communities, and Google Colab for free GPU access. **Reproducibility:** All experiments used Colab free-tier T4 GPUs, March 3–12, 2026. The harness, 24 trace files, and dataset (476×16) are at the project repository (redacted). **AI Disclosure:** AI-assisted tools were used for LaTeX formatting, code analysis, and iterative manuscript refinement; all experimental design, data collection, qualitative coding, and scientific interpretation were performed by the author.

References

[1] M. Abdin, J. Aneja, H. Awadalla, et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.

[2] W. Chen, Z. Liu, Y. Wang, et al. Quantization hurts reasoning? a systematic evaluation of compressed reasoners. In *Conference on Language Modeling (COLM)*, 2025.

[3] Z. Chen, W. Du, W. Zhang, K. Liu, J. Liu, M. Zheng, J. Zhuo, S. Zhang, D. Lin, K. Chen, and F. Zhao. T-Eval: Evaluating the tool utilization capability of large language models step by step. *arXiv preprint arXiv:2312.14033*, 2023.

[4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332, 2022.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36, 2023.

[6] Y. Dong, J. Li, J. Shang, et al. ACBench: Evaluating compression methods for LLM agents. In *International Conference on Machine Learning (ICML)*, 2025.

[7] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh. AQLM: Extreme compression of large language models via additive quantization. *arXiv preprint arXiv:2401.06118*, 2024.

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

[9] Gemma Team, Google DeepMind. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[10] A. Grattafiori, A. Dubey, A. Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[11] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *International Conference on Learning Representations*, 2024.

[12] J. Huang and K. C.-C. Chang. Towards reasoning in large language models: A survey. *Findings of the Association for Computational Linguistics: ACL*, 2023.

[13] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. Mahoney, and K. Keutzer. SqueezeLLM: Dense-and-sparse quantization. *International Conference on Machine Learning*, 2024.

[14] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *Machine Learning and Systems (MLSys)*, 2024.

[15] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, et al. AgentBench: Evaluating LLMs as agents. *International Conference on Learning Representations*, 2024.

[16] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.

[17] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *International Conference on Learning Representations*, 2024.

[18] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. Qwen3.5-4B released March 2026.

[19] T. B. Richards. AutoGPT: An autonomous GPT-4 experiment. `https://github.com/Significant-Gravitas/AutoGPT`, 2023.

[20] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024.

[21] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo. Omniquant: Omnidirectionally calibrated quantization for large language models. *International Conference on Learning Representations*, 2024.

[22] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

[23] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.

[24] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.

[25] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

[26] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shi, J. Lu, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

[27] X. Xu, M. Zhou, C. Qin, et al. A survey on knowledge distillation of large language models. *arXiv preprint arXiv:2402.13116*, 2024.

[28] J. Yang, C. E. Jimenez, A. Wettig, K. Liber, S. Y. Narasimhan, and Karthik. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[29] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[30] H. Zhang, P. Li, Z. Wu, et al. Small language models for agentic systems: A comprehensive survey. *arXiv preprint arXiv:2503.01234*, 2025.

[31] P. Zhang, G. Zeng, T. Wang, and W. Lu. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.

[32] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, et al. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.